

Machine Learning (机器学习)

Shuyuan Yang, Zhixi Feng

邮箱:syyang@xidianaedu.cn

zxfeng@xidianaedu.cn

审查

第一讲:机器学习简介

课程介绍

什么是机器学习?机器学习的应用|机器学习
的组成部分|机器学习与其他领域

第二讲:学习预测/分类 |感知器假设集|超平面/线性分类器

日常生活所需:食物、衣物、住房、交通



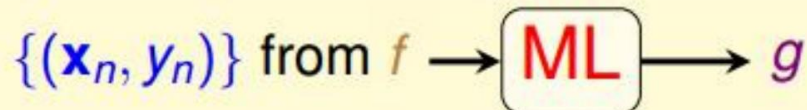
- 1 Food (Sadilek et al., 2013)
 - data: Twitter data (words + location)
 - skill: tell food poisoning likeliness of restaurant properly
- 2 Clothing (Abu-Mostafa, 2012)
 - data: sales figures + client surveys
 - skill: give good fashion recommendations to clients
- 3 Housing (Tsanas and Xifara, 2012)
 - data: characteristics of buildings and their energy load
 - skill: predict energy load of other buildings closely
- 4 Transportation (Stallkamp et al., 2012)
 - data: some traffic sign images and meanings
 - skill: recognize traffic signs accurately

ML is everywhere!

将学习问题形式化

Basic Notations

- input: $\mathbf{x} \in \mathcal{X}$ (customer application)
- output: $y \in \mathcal{Y}$ (good/bad after approving credit card)
- unknown pattern to be learned $\Leftrightarrow$ target function:
 $f: \mathcal{X} \rightarrow \mathcal{Y}$ (ideal credit approval formula)
- data $\Leftrightarrow$ training examples: $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$
(historical records in bank)
- hypothesis $\Leftrightarrow$ skill with hopefully good performance:
 $g: \mathcal{X} \rightarrow \mathcal{Y}$ ('learned' formula to be used)



机器学习与数据挖掘

Machine Learning

use data to compute hypothesis g
that approximates target f

Data Mining

use **(huge)** data to **find property**
that is interesting

- if 'interesting property' **same as** 'hypothesis that approximate target'
— **ML = DM** (usually what KDDCup does)
- if 'interesting property' **related to** 'hypothesis that approximate target'
— **DM can help ML, and vice versa** (often, but not always)
- traditional DM also focuses on **efficient computation in large database**

difficult to distinguish ML and DM in reality

简单假设集:感知器

age	23 years
annual salary	NTD 1,000,000
year in job	0.5 year
current debt	200,000

- For $\mathbf{x} = (x_1, x_2, \dots, x_d)$ '**features of customer**', compute a weighted 'score' and

approve credit if $\sum_{i=1}^d w_i x_i > \text{threshold}$

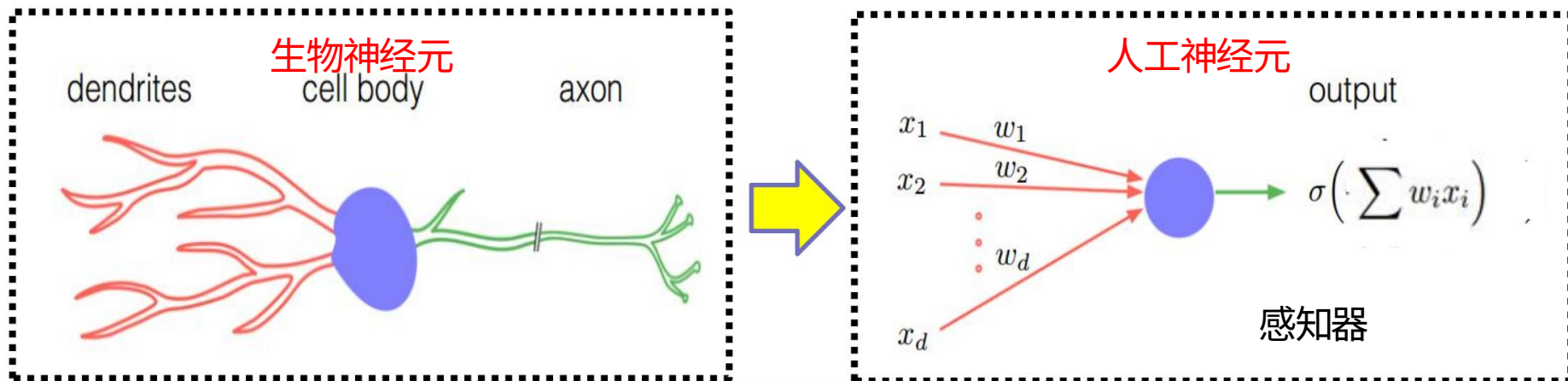
deny credit if $\sum_{i=1}^d w_i x_i < \text{threshold}$

- $\mathcal{Y}$: $\{+1(\text{good}), -1(\text{bad})\}$, 0 ignored—linear formula $h \in \mathcal{H}$ are

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - \text{threshold} \right)$$

called '**perceptron**' hypothesis historically

生物启发式感知器

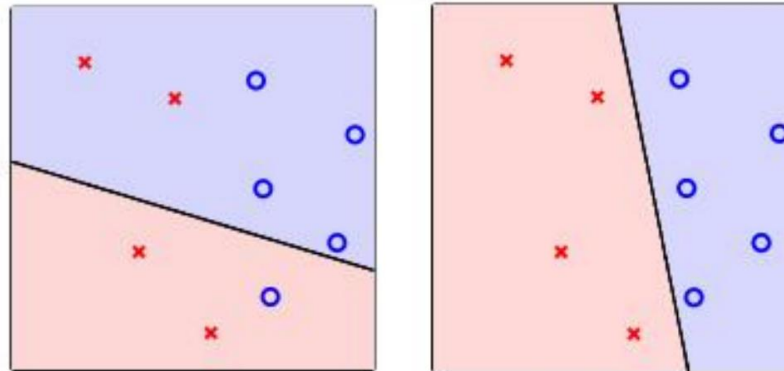


$$\begin{aligned}
 h(\mathbf{x}) &= \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - \text{threshold} \right) \\
 &= \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + \underbrace{(-\text{threshold})}_{w_0} \cdot \underbrace{(+1)}_{x_0} \right) \\
 &= \text{sign} \left(\sum_{i=0}^d w_i x_i \right) \\
 &= \text{sign} (\mathbf{w}^T \mathbf{x})
 \end{aligned}$$

感知器假设

感知器：二元分类器

$$h(\mathbf{x}) = \text{sign}(w_0 + w_1 x_1 + w_2 x_2)$$



- customer features $\mathbf{x}$: points on the plane (or points in $\mathbb{R}^d$)
- labels y : $\circ (+1)$, $\times (-1)$
- hypothesis h : **lines** (or hyperplanes in $\mathbb{R}^d$)
— **positive** on one side of a line, **negative** on the other side
- different line classifies customers differently

perceptrons $\Leftrightarrow$ **linear (binary) classifiers**

内容

第二讲:学习预测/分类|感知器假设集|超平面/线性分类器|感知器学习算法

(PLA) | PLA 的一些遗留问题|讨论:感知器与贝

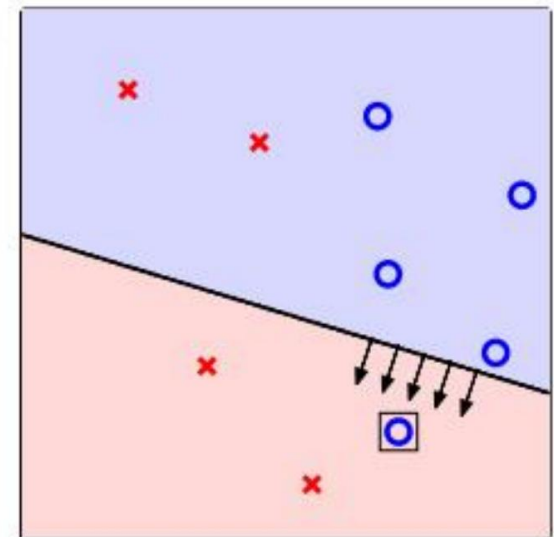
叶斯分类器| PLA 的保证

|不可分离数据|将相对“最
佳”的重量放在口袋里

从H中选择g

$\mathcal{H}$ = all possible perceptrons, $g = ?$

- want: $g \approx f$ (hard when f unknown)
- almost necessary: $g \approx f$ on $\mathcal{D}$, ideally $g(\mathbf{x}_n) = f(\mathbf{x}_n) = y_n$
- difficult: $\mathcal{H}$ is of **infinite** size
- idea: start from some g_0 , and 'correct' its mistakes on $\mathcal{D}$



will represent g_0 by its weight vector $\mathbf{w}_0$

感知器:损失函数

训练样本:

$D = \{x_i, y_i; i = 1, \dots, N\}$

经验风险 (误差):

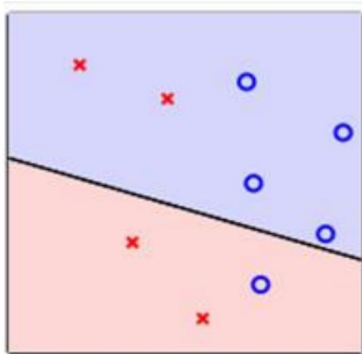
$$h = \frac{1}{N} \sum_{i=1}^N \ell(x_i, y_i, h)$$

- want: $g \approx f$ (hard when f unknown)
- almost necessary: $g \approx f$ on $\mathcal{D}$, ideally $g(\mathbf{x}_n) = f(\mathbf{x}_n) = y_n$
- difficult: $\mathcal{H}$ is of **infinite** size

$$h(\mathbf{x}) = \text{sign}(w_0 + w_1 x_1 + w_2 x_2)$$

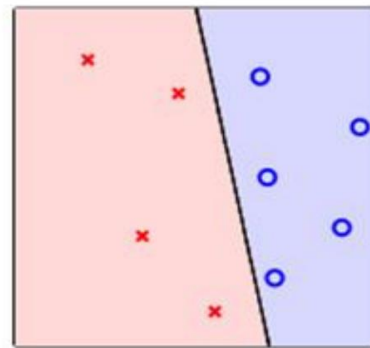
$$\min_h h$$

不可微



?

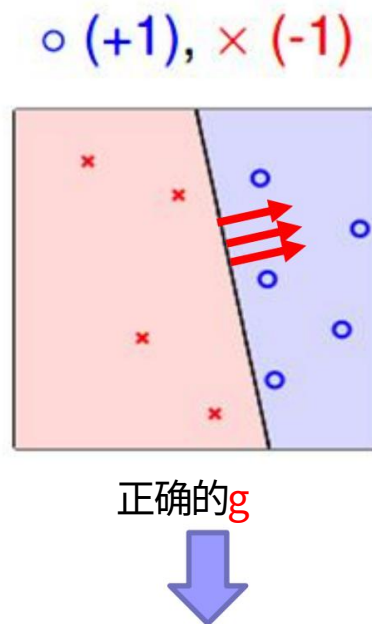
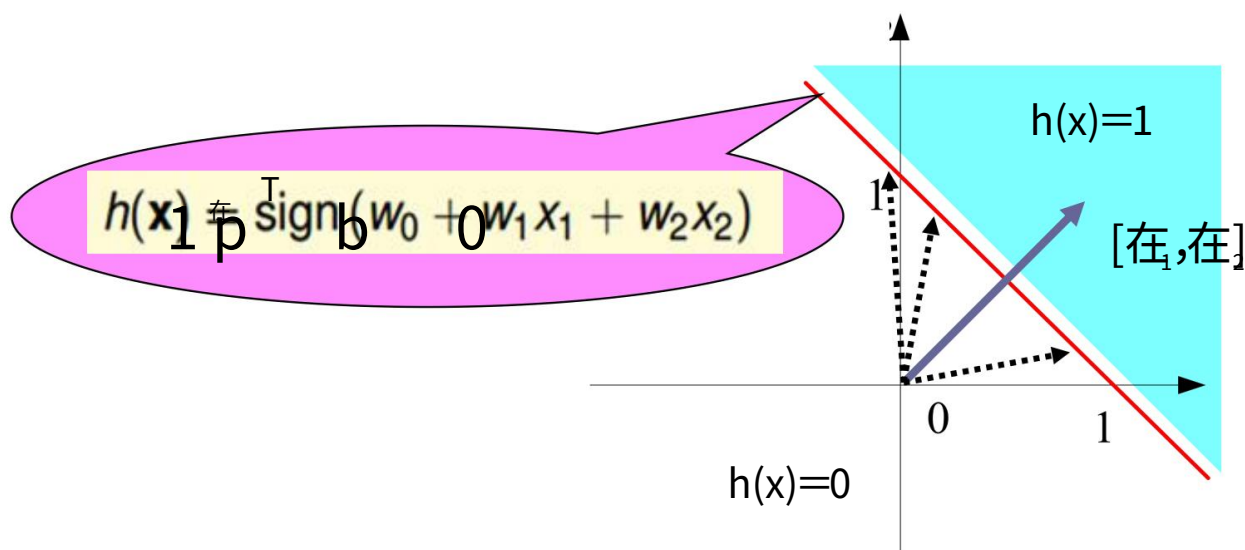
学习算法



如何找到关于 D 的假设

$$g \approx f$$

我们可以利用感知器的特性来猜测 g 的值：



1. 每个感知器都有一个决策边界。2. 感知器只能实现二元 $w_0 + w_1x_1 + w_2x_2 = 0$ 分类。
3. 多感知器 (S) 可以将数据分类成簇。

S

一个例子:猜一个正确的

克

我们可以利用感知器的特性来猜测g的值:

$$x_1 p_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}_1 \quad t y = 0_1$$

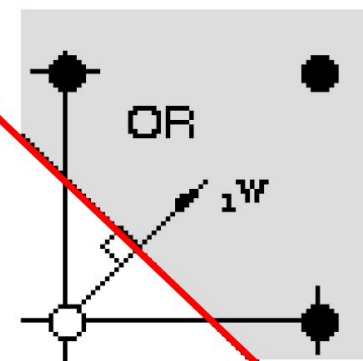
$$x_2 p_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}_2 \quad t y = 1_2$$

$$x_3 p_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}_3 \quad t y = 1_3$$

$$x_4 p_4 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}_4 \quad t y = 0_4$$

$$\lambda = \begin{matrix} \text{在}_1 \\ \text{在}_1 \\ \text{在}_2 \end{matrix} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

观察



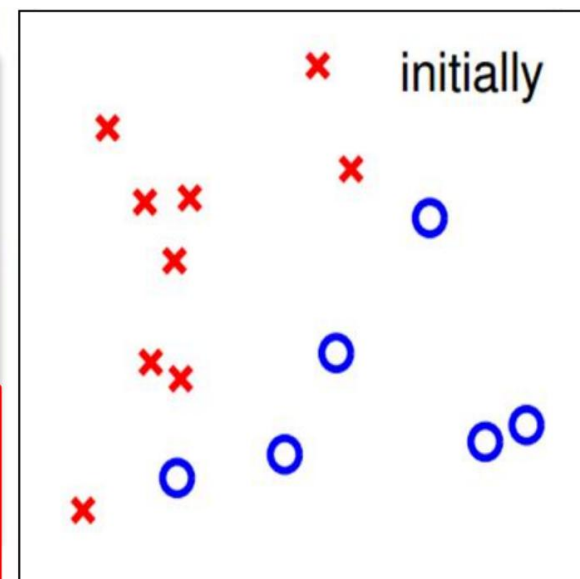
$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

$$w_1 x_1 + w_2 x_2 + b = 0 \quad \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} \quad w_1 = 0.25 + b = 0 \quad b = -0.25$$

感知器学习算法

$\mathcal{H}$ = all possible perceptrons, $g = ?$

- want: $g \approx f$ (hard when f unknown)
- almost necessary: $g \approx f$ on $\mathcal{D}$, ideally $g(\mathbf{x}_n) = f(\mathbf{x}_n) = y_n$
- difficult: $\mathcal{H}$ is of **infinite** size
- idea: start from some g_0 , and 'correct' its mistakes on $\mathcal{D}$

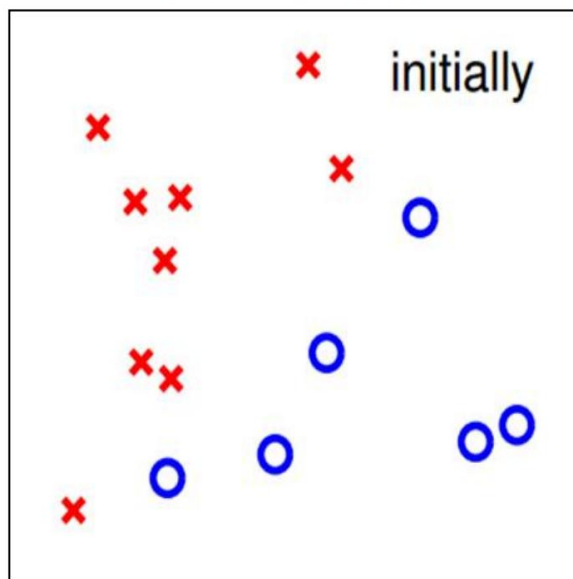


$\circ (+1), \times (-1)$

规则是什么？

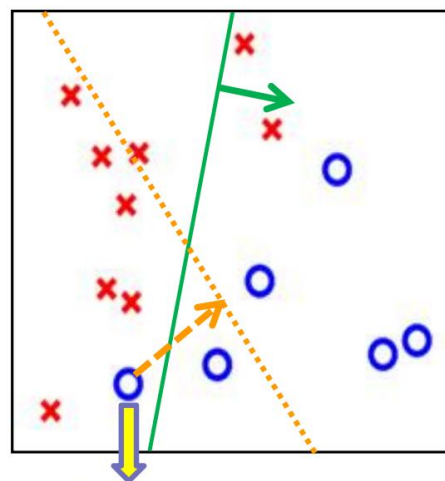
极端规则

○ (+1), × (-1)

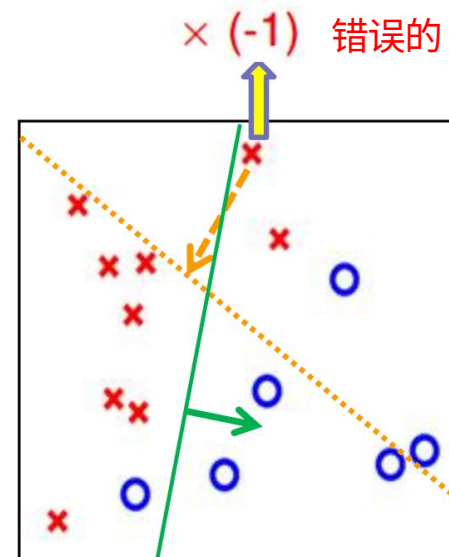


如果+1的标签错误----- $w=x$

如果-1的标签错误----- $w=-x$



○ (+1) 错误的



× (-1) 错误的

妥协的规则:解放军

start from some $\mathbf{w}_0$ (say, $\mathbf{0}$), and 'correct' its mistakes on $\mathcal{D}$

For $t = 0, 1, \dots$

- 1 find a **mistake** of $\mathbf{w}_t$ called $(\mathbf{x}_{n(t)}, y_{n(t)})$

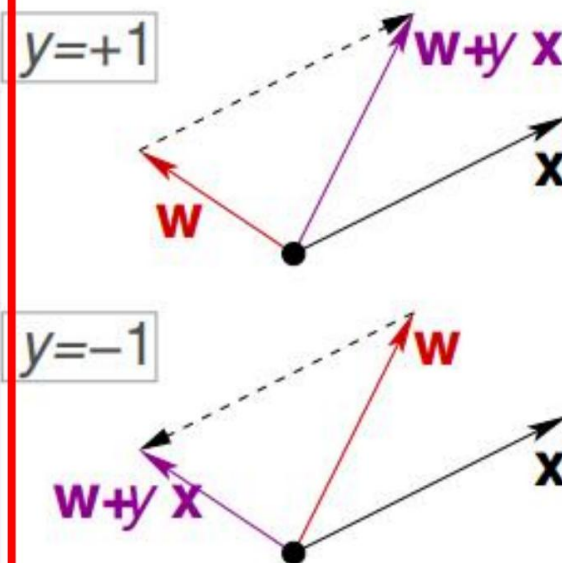
$$\text{sign}(\mathbf{w}_t^T \mathbf{x}_{n(t)}) \neq y_{n(t)}$$

- 2 (try to) correct the mistake by

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_{n(t)} \mathbf{x}_{n(t)}$$

... until **no more mistakes**

return **last $\mathbf{w}$** (called $\mathbf{w}_{\text{PLA}}$) as g



感知器学习算法 (PLA)

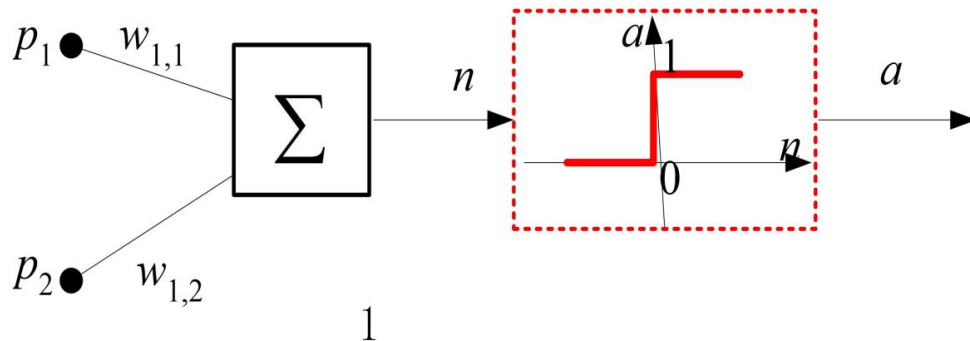
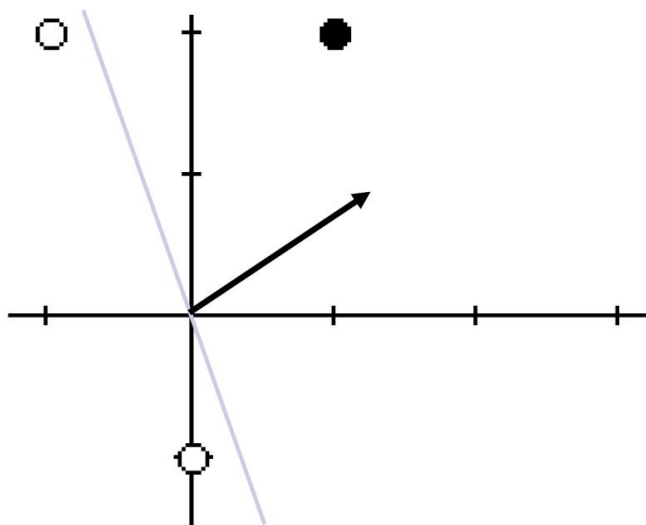
PLA 的一个例子

$$\{p_1, p_2, \dots, p_Q\}$$

$$p_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad t_1 = 1$$

$$p_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix} \quad t_2 = 0$$

$$p_3 = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \quad t_3 = 0$$

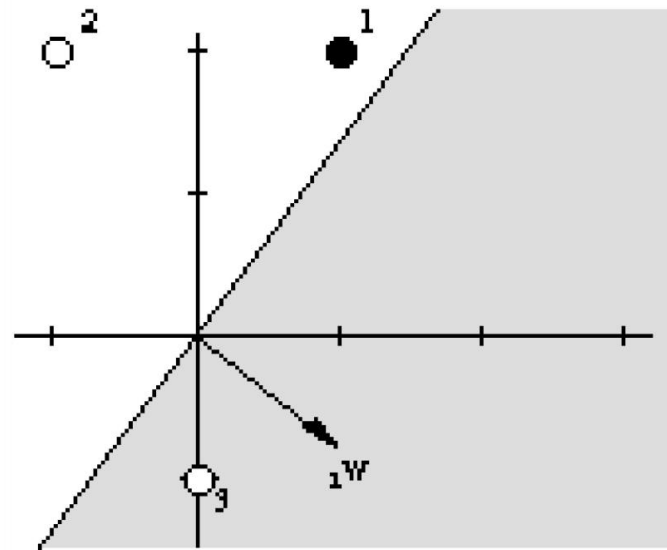


$$a = \text{hardlim}(n) = \text{hardlim}(Wp)$$

PLA 的一个例子

初始化:

$$w_1 = \begin{bmatrix} 1.0 \\ 0.8 \end{bmatrix}$$



第一个输入样本:

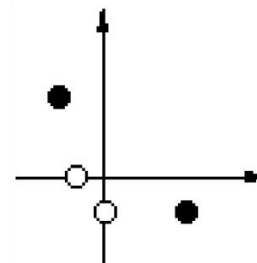
$$a = \text{hardlim}(\sum_{p=1}^P w_p x_p) = \text{hardlim}(1.0 - 0.8) = \text{hardlim}(0.2) = 1$$

×

PLA 的一个例子

$1w p1 -$, $\bar{}$ 相反, 不稳定.

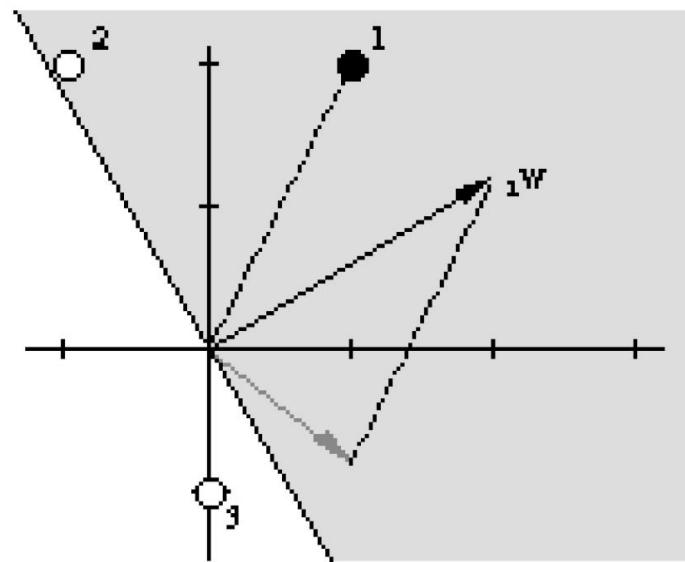
将 $p1$ 添加到 $1w$



Learning rule: 如果 $t = 1$ 和 0 , 然后 $\bar{}$ $1w$

$$\text{新的} = w_{\text{旧}} + p$$

$$1 \text{ 新的} = 1 \text{ 旧}w + p1 = \begin{bmatrix} 1.0 \\ 0.8 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2.0 \\ 1.2 \end{bmatrix}$$



PLA 的一个例子

第二个输入样本:

一个硬核 $\bar{p}_2^T$ $\text{hardlim}_{1.2} \begin{bmatrix} 2.0 \\ -1 \end{bmatrix}$

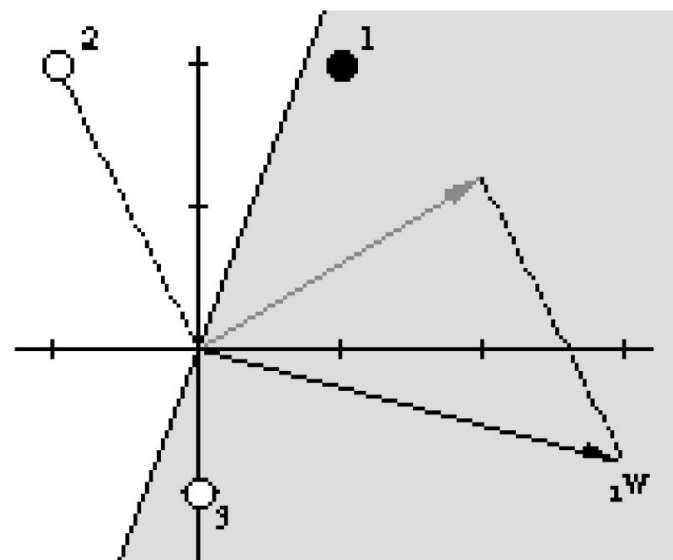
$$a = \text{hardlim}(0.4) = 1$$

(incorrect)

Rule:

如果 t 为 0 且 a 为 1, 则 $w_{\text{新的}} = w_{\text{旧}} - p$

$$w_{\text{新的}} = w_{\text{旧}} - p = \begin{bmatrix} 2.0 \\ 1.2 \end{bmatrix} - \begin{bmatrix} -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3.0 \\ 0.8 \end{bmatrix}$$



PLA 的一个例子

第三个输入样本:

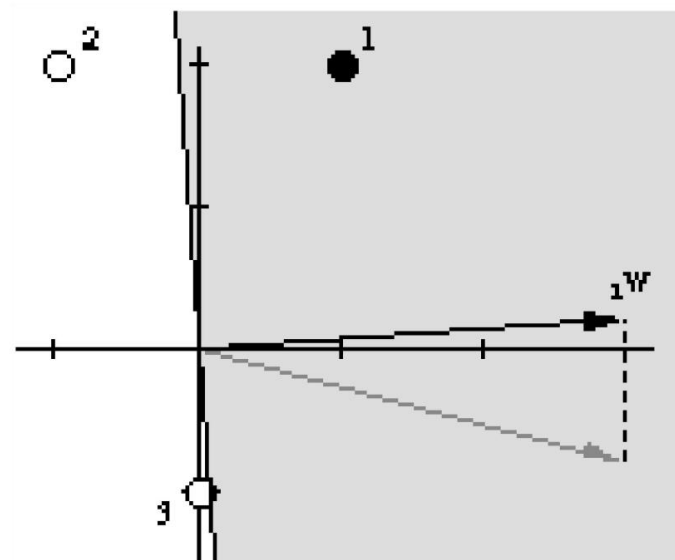
$$\begin{bmatrix} 3.0 \\ -0.8 \\ 1 \end{bmatrix}^T \quad a = \text{hardlim}(w \cdot \text{hardlim} \left[\begin{bmatrix} 0 \\ -1 \end{bmatrix} \right])$$

$a = \text{如果 } \text{rdlim}(0.8) = 1$

(incorrect)

$$\begin{aligned} \text{新的 } w_1 &= \text{老 } w_1 - d \cdot p_3 = \begin{bmatrix} 3.0 \\ -0.8 \end{bmatrix} - \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 3.0 \\ 0.2 \end{bmatrix} \end{aligned}$$

Correct



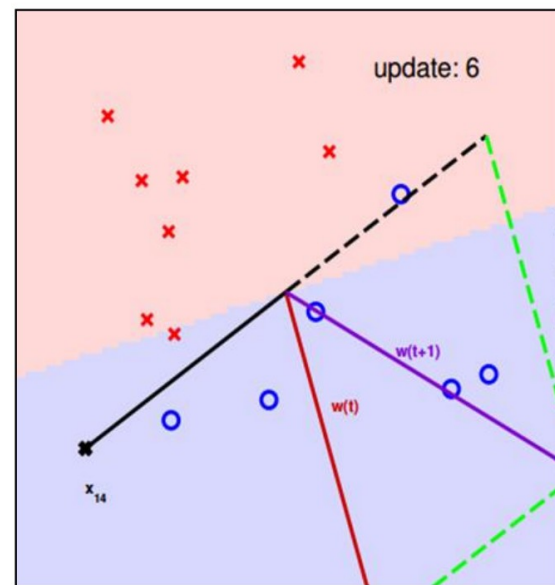
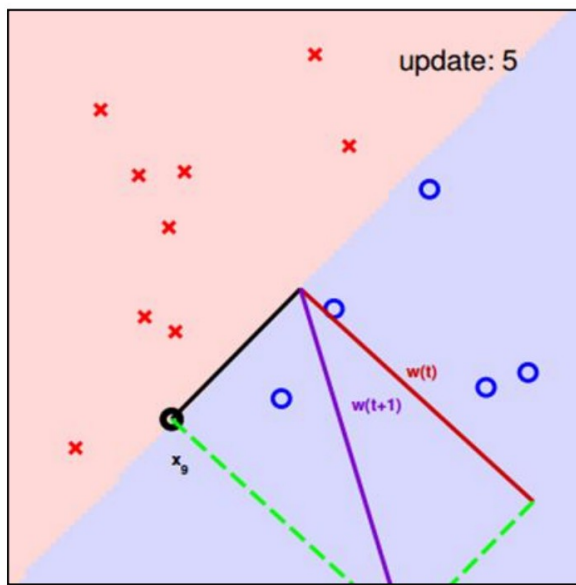
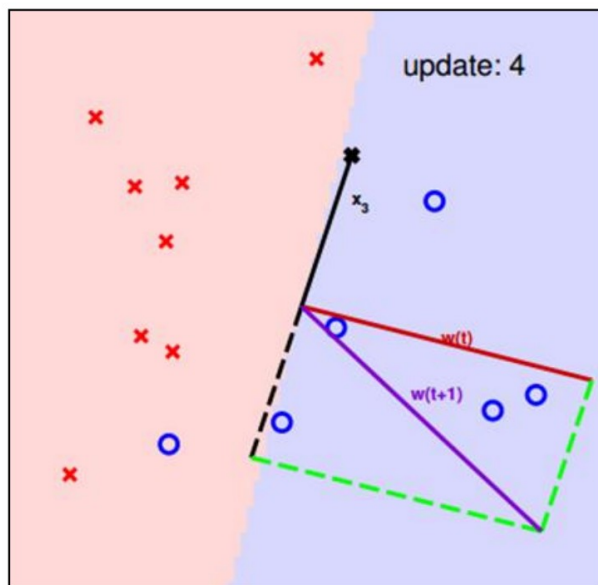
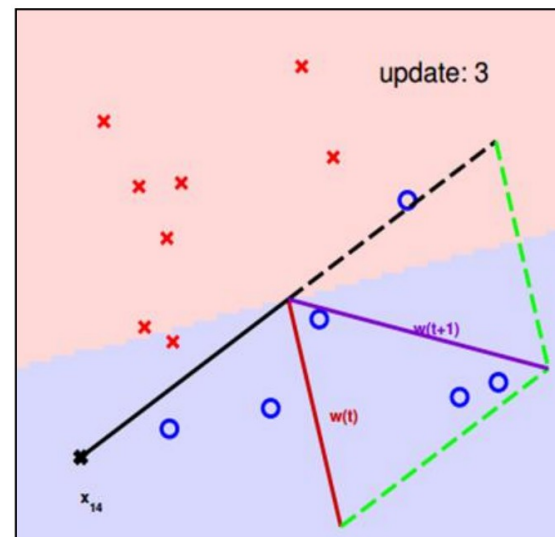
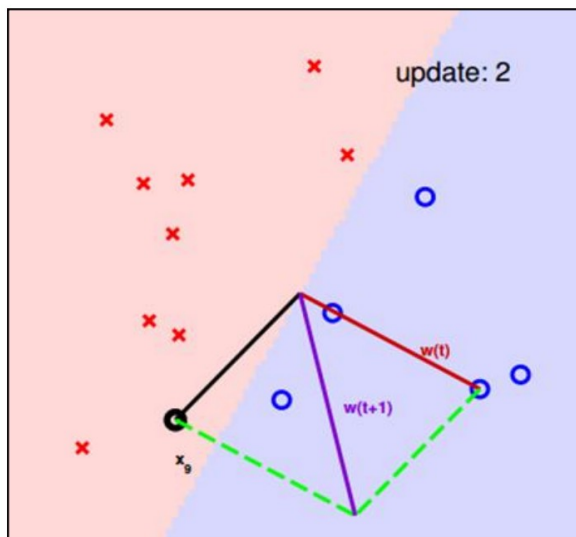
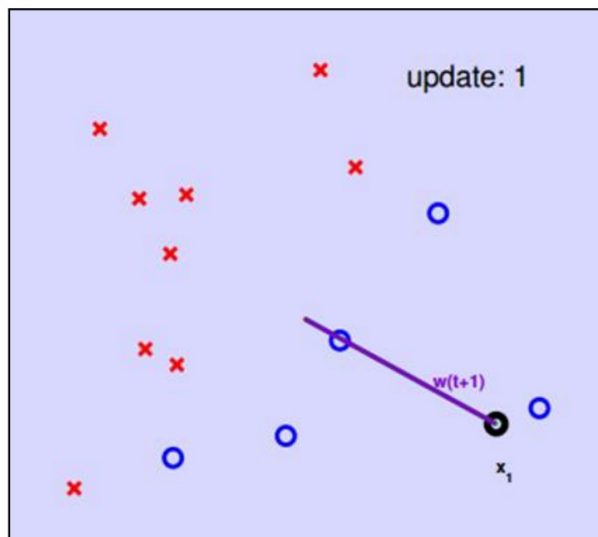
如果 $t = a$, 然后

新的 $w_1 =$ 老 w_1 。

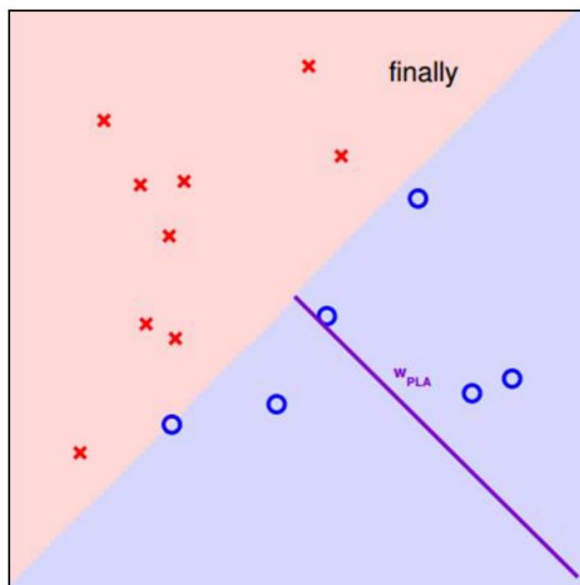
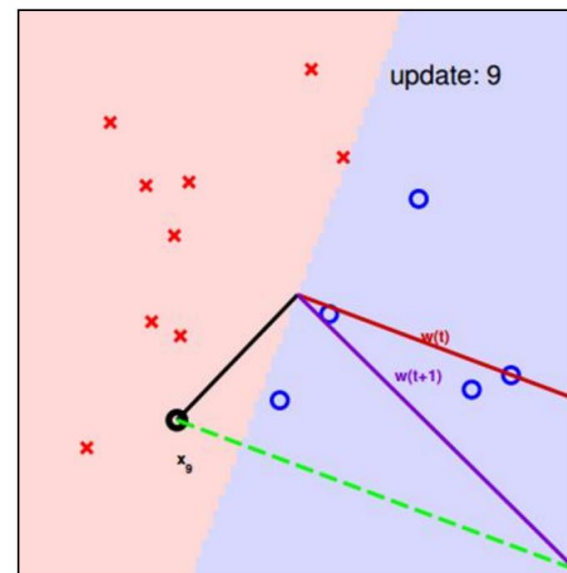
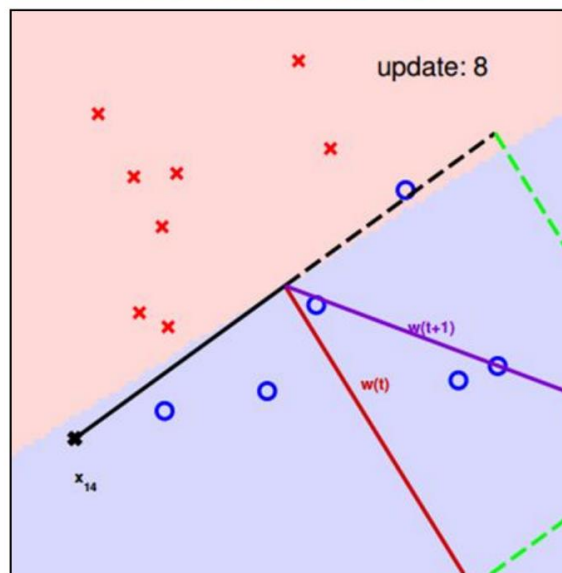
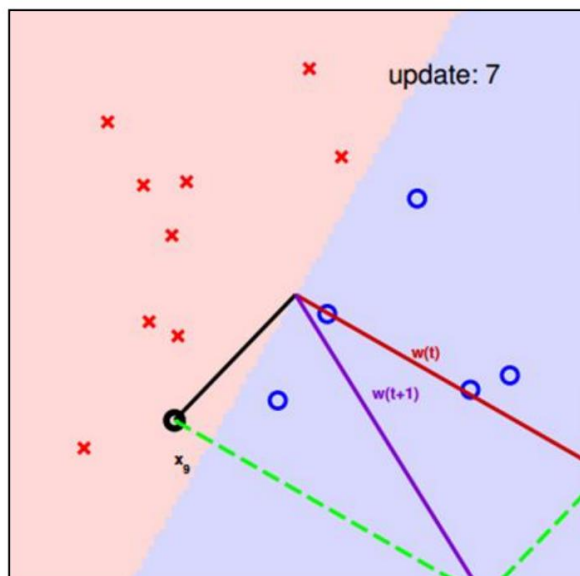
PLA 的一个例子

$$\begin{array}{c}
 p_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} t_1 = 1 \\
 p_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix} t_2 = 0 \\
 p_3 = \begin{bmatrix} 0 \\ -1 \end{bmatrix} t_3 = 0
 \end{array}
 \left\{ \begin{array}{l} \text{在} \begin{array}{c} 3.0 \\ 0.2 \end{array} \\ a = \text{hardlim}(Wt_p + b) \end{array} \right.
 \left\{ \begin{array}{l} a = \text{hardlim}(3.4) = 1 \\ a = \text{hardlim}(-2.6) = 0 \\ a = \text{hardlim}(-0.2) = 0 \end{array} \right.$$

○ (+1), × (-1)



○ (+1), × (-1)



解放军遗留的一些问题：

- ① **实际应用**是什么？
PLA 的阶数是多少？ x 的阶数是多少（朴素，PLA中随机或其他方式？
- ② **解放军**会在达到“足够”的数量后停止扩张吗？
所有 D 的修正值是多少？
- ③ PLA是否提供**保证**？
收敛性？（有多少个 PLA需要修正吗？

问题1:PLA的实际应用

start from some $\mathbf{w}_0$ (say, $\mathbf{0}$), and 'correct' its mistakes on $\mathcal{D}$

Cyclic PLA

For $t = 0, 1, \dots$

- 1 find **the next** mistake of $\mathbf{w}_t$ called $(\mathbf{x}_{n(t)}, y_{n(t)})$

$$\text{sign} \left(\mathbf{w}_t^T \mathbf{x}_{n(t)} \right) \neq y_{n(t)}$$

- 2 correct the mistake by

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_{n(t)} \mathbf{x}_{n(t)}$$

... until **a full cycle of not encountering mistakes**

next can follow naïve cycle $(1, \dots, N)$
or precomputed random cycle

Q2:解放军会“停止”吗?

‘correct’ mistakes on $\mathcal{D}$ **until no mistakes**

Algorithmic: halt (with no mistake)?

- naïve cyclic: ??
- random cyclic: ??
- other variant: ??

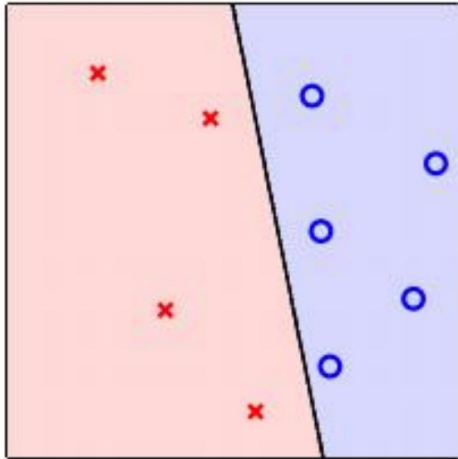
Learning: $g \approx f$?

- on $\mathcal{D}$, if halt, yes (no mistake)
- outside $\mathcal{D}$: ??
- if not halting: ??

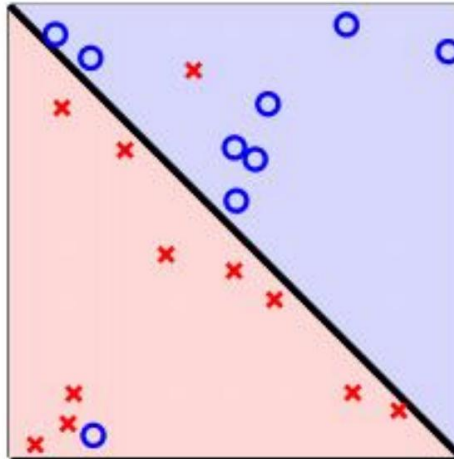
[to be shown] if (...), after ‘enough’ corrections,
any PLA variant halts

Q2:解放军会“停止”吗?

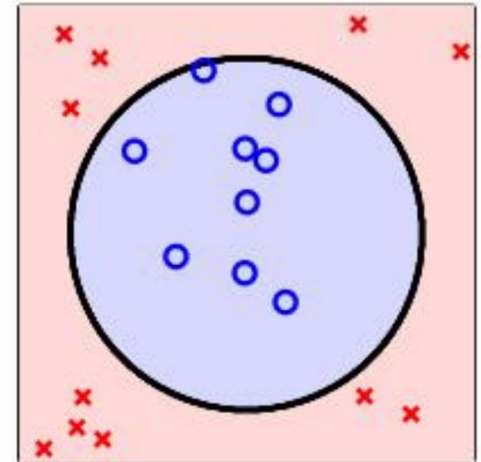
- if PLA halts (i.e. no more mistakes),
(**necessary condition**) $\mathcal{D}$ allows some $\mathbf{w}$ to make no mistake
- call such $\mathcal{D}$ **linear separable**



(linear separable)



(not linear separable)

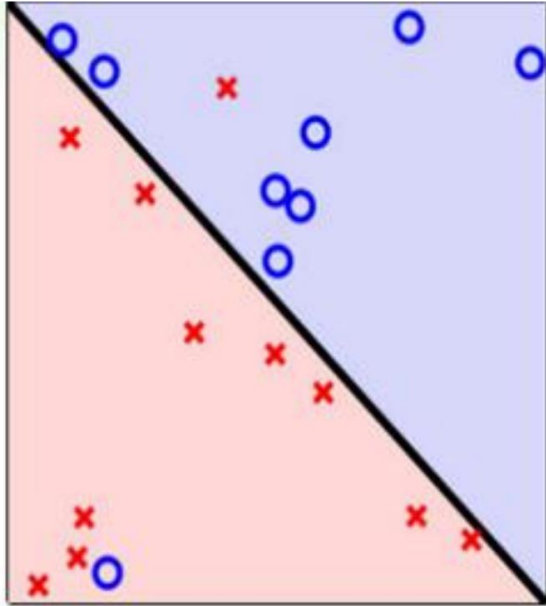


(not linear separable)

assume linear separable $\mathcal{D}$,
does PLA always **halt**?

是的

Q2:解放军会“停止”吗?



- assume 'little' noise: $y_n = f(\mathbf{x}_n)$ **usually**
- if so, $g \approx f$ on $\mathcal{D} \Leftrightarrow y_n = g(\mathbf{x}_n)$ **usually**
- how about

$$\mathbf{w}_g \leftarrow \operatorname{argmin}_{\mathbf{w}} \sum_{n=1}^N \mathbb{I}[y_n \neq \operatorname{sign}(\mathbf{w}^T \mathbf{x}_n)]$$

—**NP-hard to solve, unfortunately**

can we **modify PLA** to get
an 'approximately good' g ?

Q2:解放军会“停止”吗?

modify PLA algorithm (black lines) by **keeping best weights in pocket**

initialize pocket weights $\hat{w}$

For $t = 0, 1, \dots$

- ① find a (random) mistake of w_t called $(x_{n(t)}, y_{n(t)})$
- ② (try to) correct the mistake by

$$w_{t+1} \leftarrow w_t + y_{n(t)} x_{n(t)}$$

- ③ **if w_{t+1} makes fewer mistakes than $\hat{w}$, replace $\hat{w}$ by w_{t+1}**

...until **enough iterations**

return **$\hat{w}$ (called w_{POCKET}) as g**

a simple modification of PLA to find
(somewhat) 'best' weights

欢乐时光

Should we use pocket or PLA?

Since we do not know whether $\mathcal{D}$ is linear separable in advance, we may decide to just go with pocket instead of PLA. If $\mathcal{D}$ is actually linear separable, what's the difference between the two?

- 1 pocket on $\mathcal{D}$ is slower than PLA
- 2 pocket on $\mathcal{D}$ is faster than PLA
- 3 pocket on $\mathcal{D}$ returns a better g in approximating f than PLA
- 4 pocket on $\mathcal{D}$ returns a worse g in approximating f than PLA

欢乐时光

Should we use pocket or PLA?

Since we do not know whether $\mathcal{D}$ is linear separable in advance, we may decide to just go with pocket instead of PLA. If $\mathcal{D}$ is actually linear separable, what's the difference between the two?

- ① pocket on $\mathcal{D}$ is slower than PLA
- ② pocket on $\mathcal{D}$ is faster than PLA
- ③ pocket on $\mathcal{D}$ returns a better g in approximating f than PLA
- ④ pocket on $\mathcal{D}$ returns a worse g in approximating f than PLA

Reference Answer: ①

Because pocket need to check whether $\mathbf{w}_{t+1}$ is better than $\hat{\mathbf{w}}$ in each iteration, it is slower than PLA. On linear separable $\mathcal{D}$, $\mathbf{w}_{\text{POCKET}}$ is the same as $\mathbf{w}_{\text{PLA}}$, both making no mistakes.

Q3: PLA的保证

给 $x_1, y_1, x_2, y_2, \dots, x_Q, y_Q$

输出 $a \quad \text{hard lim}(w_1^T x + b) \quad b)$

让

在 b , x_q x_q
1

$$n = w p^T + b = w x^T$$

新的 w 老的 w $e x$



w_k w_k 1 $x(k-1)$



$x_k (x_k): x_1 x_2 \dots$ 问, $1, 2, \dots$ 问



如果 $y_q = 1$, $w x_q^T = 0$
如果 $y_q = 0$, $w x_q^T = 0$

经过k次迭代后，

$$W_X(k) = (0 \ 1)^X \dots (k \ X_1)$$



然后

$$\mathbf{T}^T \mathbf{W} \mathbf{W}^T \mathbf{W} \mathbf{k} = \mathbf{T}^T \mathbf{W} \mathbf{k} (0) + \mathbf{T}^T \mathbf{W} \mathbf{k} (1) + \dots + \mathbf{T}^T \mathbf{W} \mathbf{k} (k-1)$$

$$\frac{\mathbf{W}^* \mathbf{X}^T}{\mathbf{W}^* \mathbf{W}^T} \mathbf{k} \quad , \quad (\mathbf{W}^T \mathbf{W} - \mathbf{I})^2 \mathbf{k} \left\|_{2, \mathbf{W} \mathbf{W}}^* \right\| \left\| \mathbf{k} \right\|^2$$

下边界

$$\| \text{在 } k) \| ^2 \frac{ (w w ^T (k) \geq ^2 }{ \| \text{在 } ^* \| ^2 } \frac{ (k) ^2 }{ \| \text{在 } ^* \| ^2 }$$

上边界

$$\|w(k)\|^2 = w^T(k) w(k)$$

$$= (w(k-1) - \eta \nabla J(w(k-1)))^T (w(k-1) - \eta \nabla J(w(k-1)))$$

$$= w^T(k-1) w(k-1) - 2\eta w^T(k-1) \nabla J(w(k-1)) + \eta^2 \nabla J^T(w(k-1)) \nabla J(w(k-1))$$

vFor

$$w^T(k-1) \nabla J(w(k-1)) \leq 0$$



$$\|w(k)\|^2 \leq \|w(k-1)\|^2 + \eta^2 \|\nabla J(w(k-1))\|^2$$

$$\|x(k)\|_2^2 \leq \|x(0)\|_2^2 + \dots + \|x(k-1)\|_2^2$$

vLet $\| \max(i) \|_2^2$

$$\|w(k)\|_2^2 \leq k \| \max(i) \|_2^2$$



$$k \| \max(i) \|_2^2 \geq \|x(k)\|_2^2 \geq \frac{\|x(k)\|_2^2}{\|x^*\|_2^2}$$



$$k \geq \frac{\|x^*\|_2^2}{2}$$

PLA 事实: $\mathbf{w}_t$ 与 $\mathbf{w}_f$ 更加一致

linear separable $\mathcal{D} \Leftrightarrow$ **exists perfect $\mathbf{w}_f$ such that $y_n = \text{sign}(\mathbf{w}_f^T \mathbf{x}_n)$**

- **$\mathbf{w}_f$ perfect** hence **every $\mathbf{x}_n$ correctly away from line:**

$$y_{n(t)} \mathbf{w}_f^T \mathbf{x}_{n(t)} \geq \min_n y_n \mathbf{w}_f^T \mathbf{x}_n > 0$$

- **$\mathbf{w}_f^T \mathbf{w}_t \uparrow$** by updating with any $(\mathbf{x}_{n(t)}, y_{n(t)})$

$$\begin{aligned} \mathbf{w}_f^T \mathbf{w}_{t+1} &= \mathbf{w}_f^T (\mathbf{w}_t + y_{n(t)} \mathbf{x}_{n(t)}) \\ &\geq \mathbf{w}_f^T \mathbf{w}_t + \min_n y_n \mathbf{w}_f^T \mathbf{x}_n \\ &> \mathbf{w}_f^T \mathbf{w}_t + 0. \end{aligned}$$

$\mathbf{w}_t$ appears more aligned with $\mathbf{w}_f$ after update
(really?)

PLA小知识:重量增长速度不会太快

$\mathbf{w}_t$ changed only when mistake

$$\Leftrightarrow \text{sign}(\mathbf{w}_t^T \mathbf{x}_{n(t)}) \neq y_{n(t)} \Leftrightarrow y_{n(t)} \mathbf{w}_t^T \mathbf{x}_{n(t)} \leq 0$$

- mistake 'limits' $\|\mathbf{w}_t\|^2$ growth, even when updating with 'longest' $\mathbf{x}_n$

$$\begin{aligned} \|\mathbf{w}_{t+1}\|^2 &= \|\mathbf{w}_t + y_{n(t)} \mathbf{x}_{n(t)}\|^2 \\ &= \|\mathbf{w}_t\|^2 + 2y_{n(t)} \mathbf{w}_t^T \mathbf{x}_{n(t)} + \|y_{n(t)} \mathbf{x}_{n(t)}\|^2 \\ &\leq \|\mathbf{w}_t\|^2 + 0 + \|y_{n(t)} \mathbf{x}_{n(t)}\|^2 \\ &\leq \|\mathbf{w}_t\|^2 + \max_n \|y_n \mathbf{x}_n\|^2 \end{aligned}$$

start from $\mathbf{w}_0 = \mathbf{0}$, after T mistake corrections,

$$\frac{\mathbf{w}_f^T}{\|\mathbf{w}_f\|} \frac{\mathbf{w}_T}{\|\mathbf{w}_T\|} \geq \sqrt{T} \cdot \text{constant}$$

关于PLA的更多信息

Guarantee

as long as **linear separable** and **correct by mistake**

- inner product of $\mathbf{w}_f$ and $\mathbf{w}_t$ grows fast; length of $\mathbf{w}_t$ grows slowly
- PLA 'lines' are more and more aligned with $\mathbf{w}_f \Rightarrow$ halts

Pros

simple to implement, fast, works in any dimension d

Cons

- **'assumes' linear separable $\mathcal{D}$** to halt
 - property unknown in advance (no need for PLA if we know $\mathbf{w}_f$)
- not fully sure **how long halting takes** (ρ depends on $\mathbf{w}_f$)
 - though practically fast

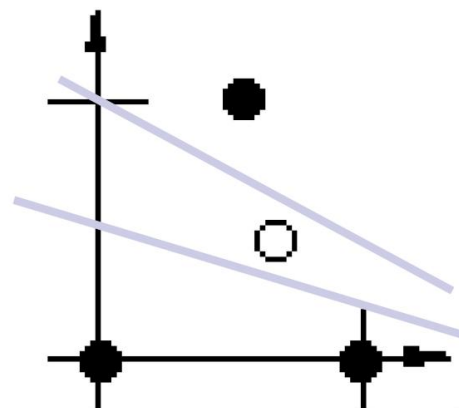
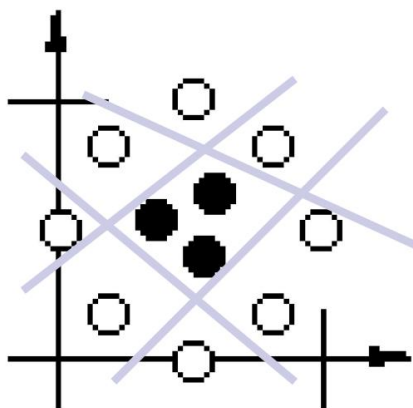
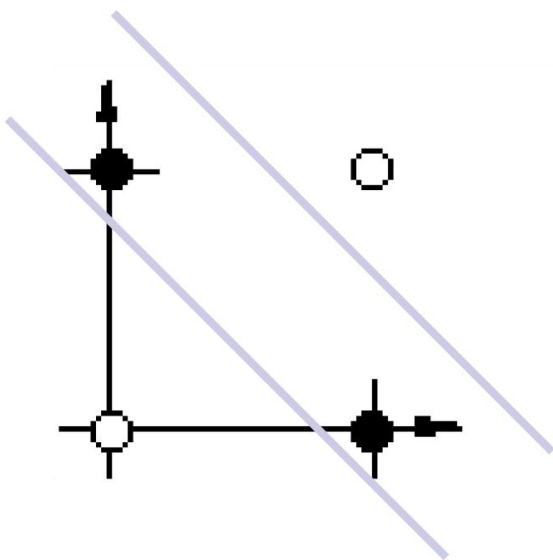
what if $\mathcal{D}$ not linear separable?

内容

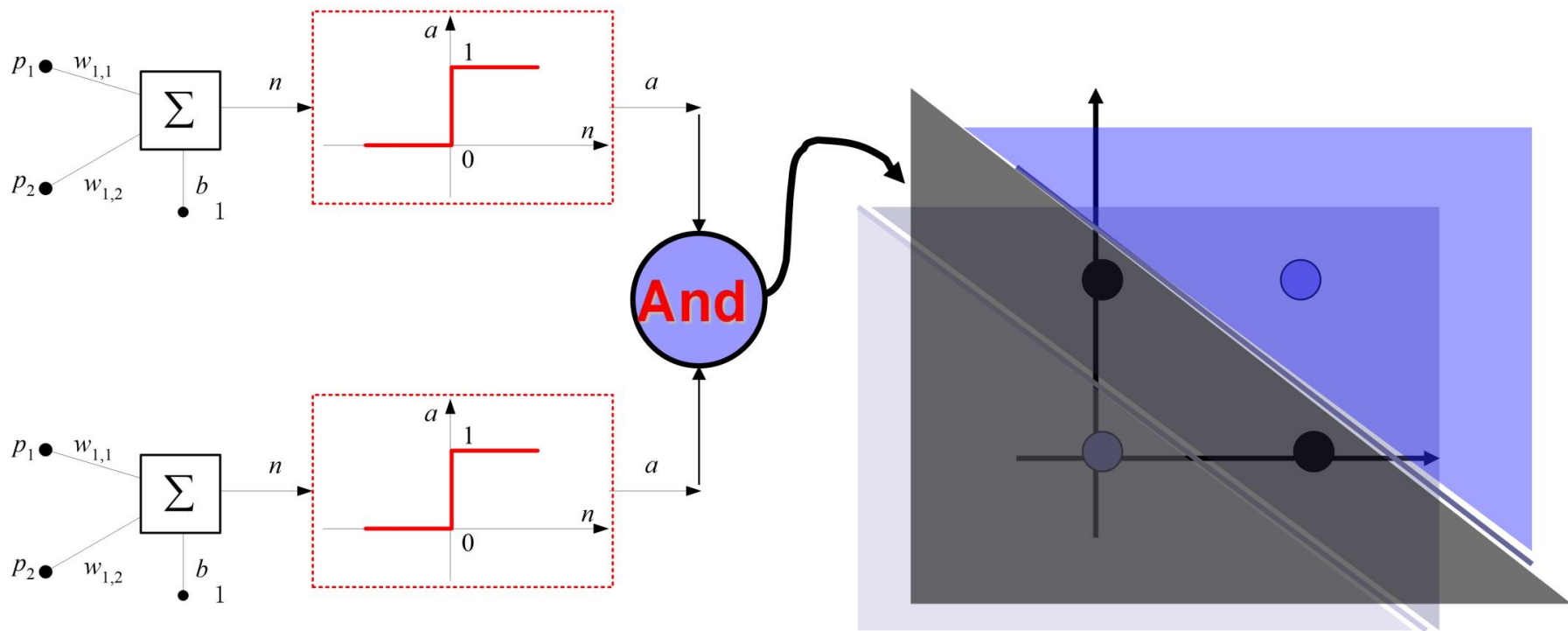
第二讲:学习预测/分类|纠正错误并迭代改进| PLA 保证

|不可分离数据|多层感知器网络
(MLPN)

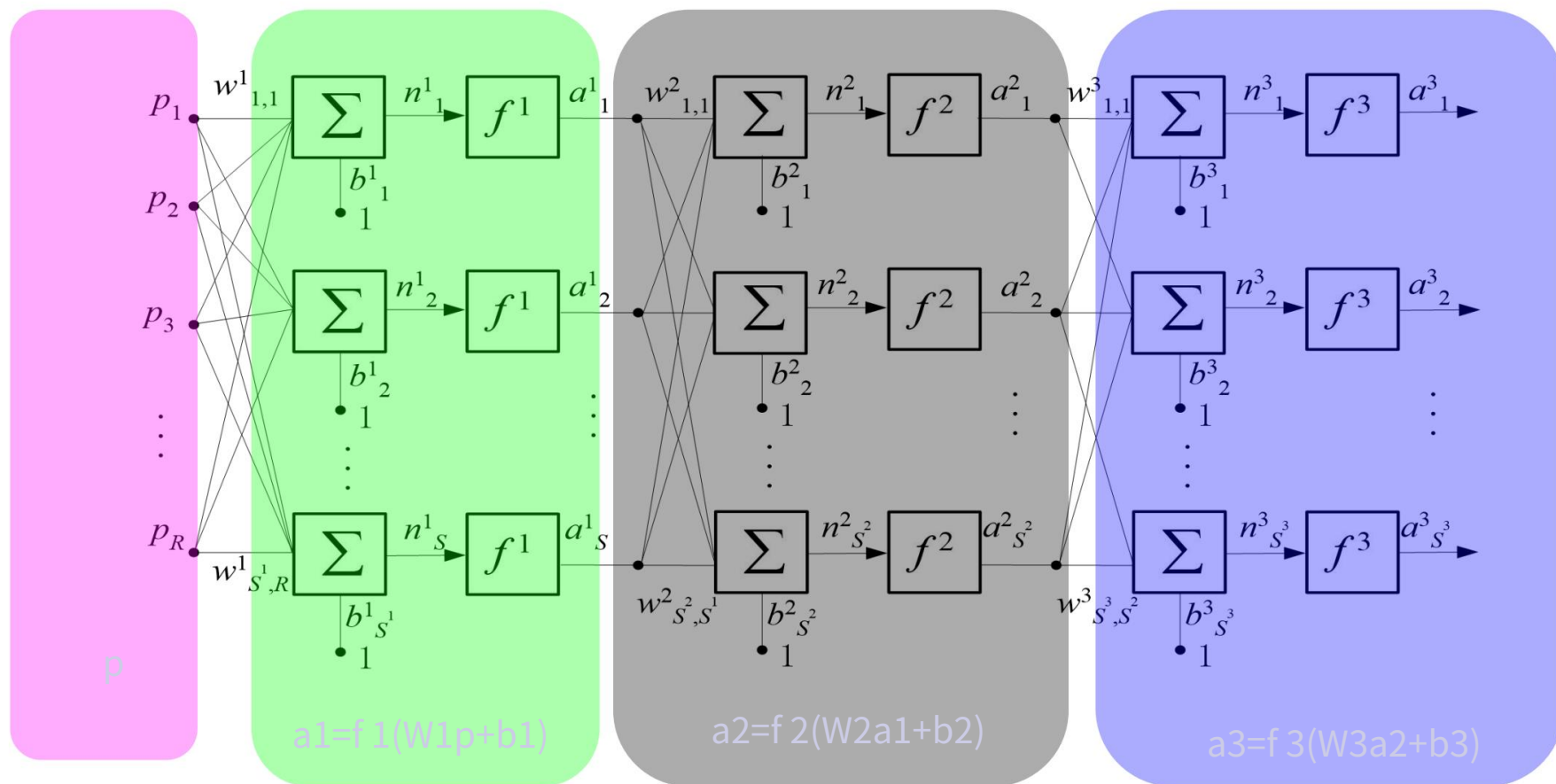
不可分离数据



不可分离数据

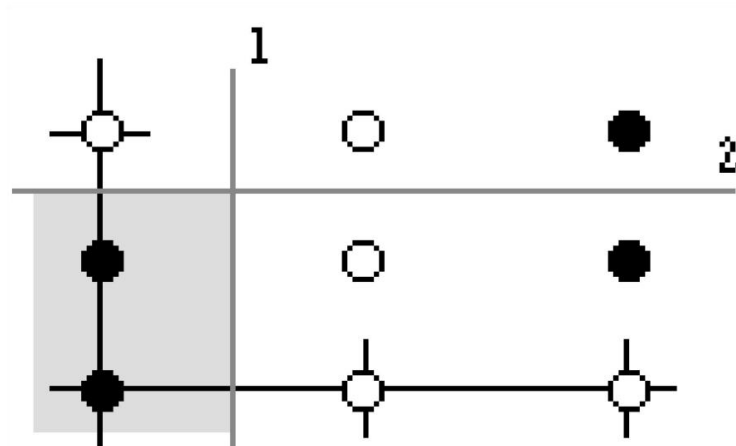


多层感知器网络



$$a3 = f3(W3f2(W2f1(W1p + b1) + b2) + b3)$$

一个例子



Boundary 1:

$$1 = \text{Hardlim}(-1.0 p + 0.5) \quad \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$$

Boundary 2:

$$1 = \text{hardlim}(0 - 1 p + 0.75) \quad \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$$

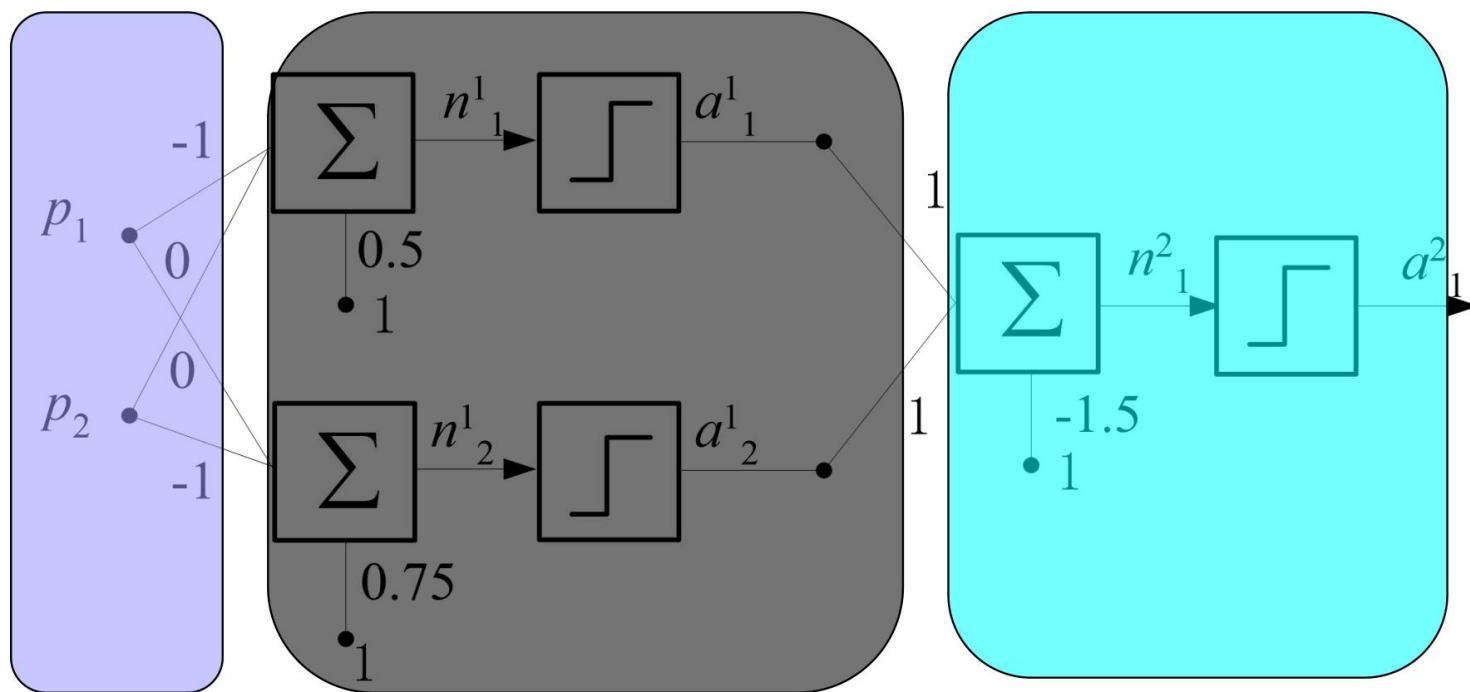
一个例子

Sub-network 1

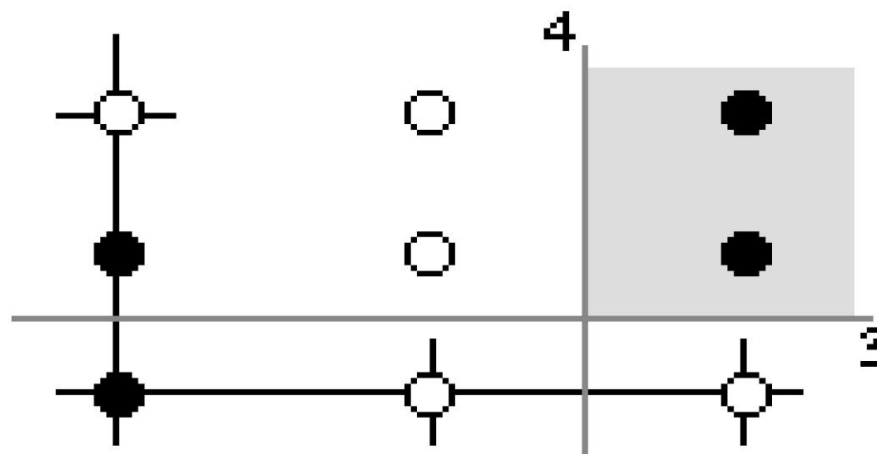
Input

Boundary

And



一个例子



Boundary 3: $1 = \text{hardlim}(1 - 0.5 p[-1.5] a_3)$

Boundary 4: $1 = \text{hardlim}(0.1 p - 0.25) a_4$

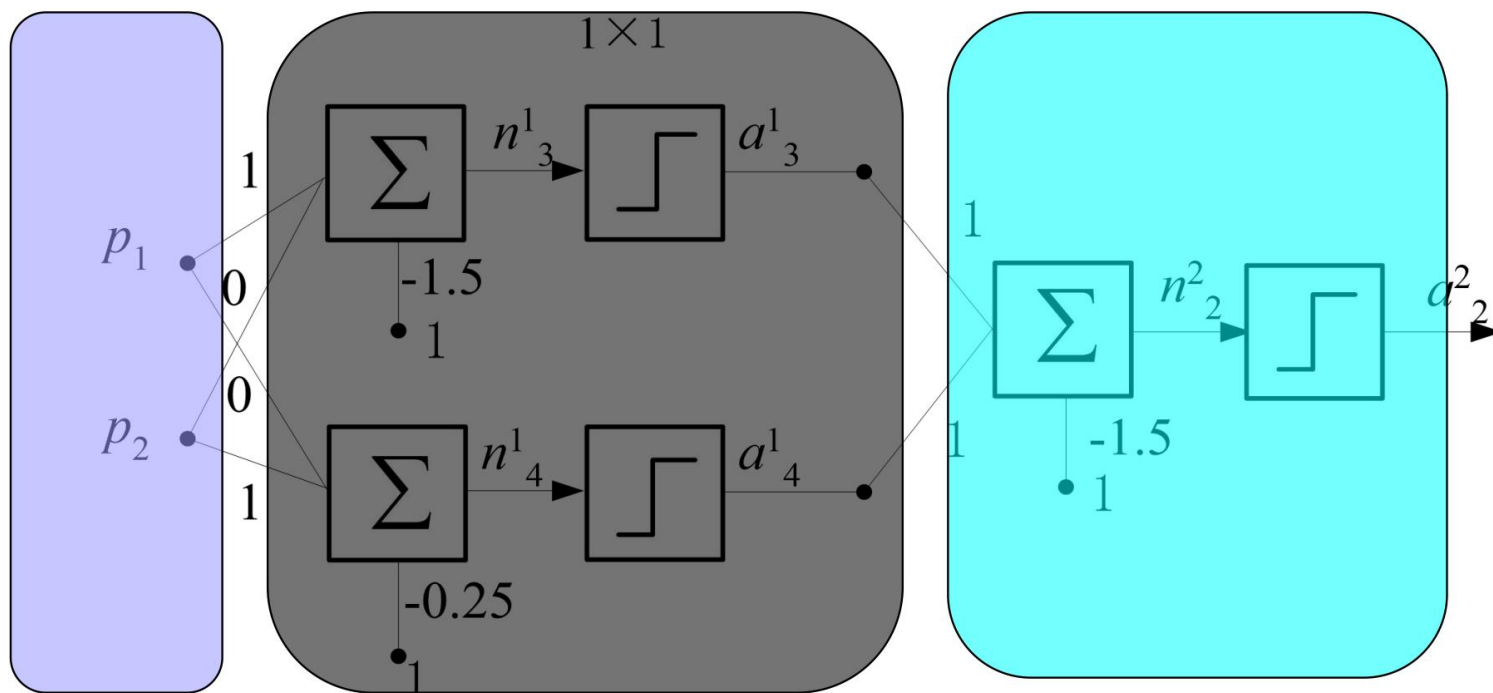
一个例子

Sub-network 2

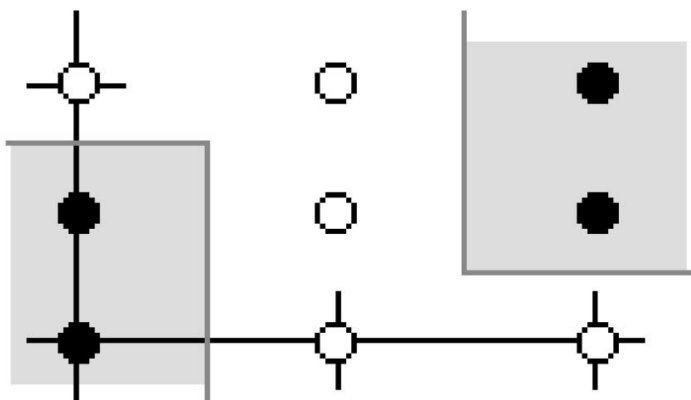
Input

Boundary

And



一个例子



$$\text{在}^1 = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$1b = \begin{bmatrix} 0.5 \\ 0.75 \\ -1.5 \\ -0.25 \end{bmatrix}$$

$$\text{在}^2 = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

$$2b = \begin{bmatrix} -1.5 \\ -1.5 \end{bmatrix}$$

$$\text{在}^3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$3b = \begin{bmatrix} 0.5 \end{bmatrix}$$

Input Boundary

And

Or

